



Software Factory en la Era de la IA

Practicas

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Coding Standards

Estándares de Codificación

Introducción

Los coding standards son un conjunto acordado de reglas que todo código dentro del proyecto debe seguir. Garantizan uniformidad y facilitan code reviews.

Naming conventions

Element	Convention	Example
Classes	PascalCase	Customer
Methods	camelCase	getUser()
Variables	camelCase	totalAmount
Constants	UPPER_SNAKE	MAX_RETRIES
private fields	_likeThis	_cache

Formatting con EditorConfig

El archivo `.editorconfig` fuerza una indentación, charset y final de línea consistentes:

```
root = true

[*]
indent_style = space
indent_size = 2
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true

[*.*ts]
indent_size = 2

[*.{json,yml,yaml}]
indent_size = 2
```

Linters y Formatters

Usar herramientas que automaticen las reglas:

Language	Linters	Formatter
Python	ruff, pylint	black
TypeScript	eslint	prettier
Go	golangci-lint	gofmt

Configura linters en CI, no solo local.

```
workflow_dispatch: (ci-validate)
```

Commits ligados a estilos

Los commits deberían pasar checks de linting. Referenciar convenciones en [Git conventions](#).

Reglas estructurales

- Una responsabilidad por fichero
- < 200 líneas en métodos
- < 50 líneas en funciones
- Prohibir `eval` y `exec`
- Prohibir imports circulares

Temas adicionales

- Excepciones: responder en todos los catch
- Lverages de lenguaje: no reinventar herramientas
- Si se añade dependencia externa, registrar razón y aprobar

Integración en CI

Todo PR falla si el linting no pasa:

```
- name: Lint
  run: ruff check .
```

Relacionado: DoD checklist.

Herramientas recomendadas

Tool	Purpose
EditorCofig	cross-editor formatting
Prettier	universal formatter
Eslint	TS/JS linter
Ruff	Python linter

Tool	Purpose
golangci-lint	Go linter

Más información

- Métricas de calidad de código
- Git hooks
- Clean Code principles

Clean Code

Clean Code

Introducción

Clean Code es un conjunto de principios enunciados por Robert C. Martin ("Uncle Bob") en su libro del mismo nombre, que definen cómo escribir código legible, mantenible y seguro.

Principios clave

1. Nombres significativos

Mal ejemplo	Buen ejemplo
<code>int d; // elapsed time</code>	<code>int elapsedTime;</code>
<code>getThem();</code>	<code>getApprovedVendors();</code>
<code>f = n % 2 == 0;</code>	<code>isEven = n % 2 == 0;</code>

- Nombres deben revelar intención.
- Evitar desinformación: hp vs hypotenuse.

2. Funciones pequeñas

Una función debe hacer una sola cosa y debe hacerla bien.

```
// Mal
public void process(List<Item> items) {
    items.forEach(item -> {
        if (item.isValid()) {
            save(item);
            send(item);
            log(item);
        }
    });
}
```

Refactor to:

```
// Mejor
public void processValidItems(List<Item> items) {
    items.stream()
        .filter(this::isValid)
```

```
        .forEach(this::processItem);  
    }
```

Reglas de funciones:

- No tener más de ~20 líneas.
- Sin argumentos de control: `setUser(isAdmin) => setAdminUser()`.
- Los argumentos booleanos son mal olor.
- Prefer exceptions a retorno de errores.

3. Comentarios

El código debe ser auto-documentado. Preferir código explicativo en lugar de comentarios.

```
// Bad  
// check if user is authorized  
if (user.role === 'admin') {}  
  
// Good  
if (hasElevatedPrivileges(user)) {}
```

La documentación vive en archivos README, ADRs, RFCs. Relacionado: Documentacion tecnica.

4. Formato

Consistencia en todo el proyecto. Usar:

- EditorConfig (ver Coding standards)
- Linter + formatter
- blank lines entre conceptos

5. Manejo de errores

```
// Bad  
public void saveUser(User u) {  
    try {  
        // do save  
    } catch (Exception e) {  
        e.log();  
    }  
}  
  
// Good  
public void saveUser(User u) throws UserSaveException {  
    // do save  
    // let call handle the exception  
}
```

6. DRY (Don't repeat)

Duplicación lleva a inconsistencia:

- Si hay duplicación, extraer a método.
- Si hay duplicación en config, externalizar.

7. Objetos y estructuras de datos

Principio	Explicación
Law of Demeter	Talk only to friends
Command-Query Segregation	Método o pregunta o com, no ambas
Tell, don't ask	Delega, no solicites info vacía

Code smells

Indicadores de que se necesita refactoring:

- **Long method**
- **Large class**
- **Duplicated code**
- **Primitive obsession**
- **Shotgun surgery**
- **Feature envy**

Los code smells no son bugs. Guían hacia refactoring oportunistas. Ver Refactoring continuo.

Relación con otras prácticas

Práctica	Relación
./01 coding standards	Base para higiene
./03 test driven development	Código limpio es más testeable
./11 definition of done	No DoD sin código limpio
./06 code review best practices	Code reviews detectan smells
../08 Calidad/READ LINT	Lint detecta olores

Conclusión

Clean Code es una disciplina, no un objetivo. Se logra con esfuerzo continuo y en equipo.

Test Driven Development

Test-Driven Development

¿Qué es TDD?

TDD (Test-Driven Development) te pide escribir la prueba antes del código de producción. Sigue el ciclo red-green-refactor:

RED -> GREEN -> REFACTOR

1. **RED**: Escribir un test fallido que describa el comportamiento deseado.
2. **GREEN**: Escribir el mínimo código necesario para pasar el test.
3. **REFACTOR**: Mejorar el código sin cambiar el comportamiento.

```
# RED step
def test_should_return_twenty_celsius():
    f = FahrenheitWeather()
    assert f.to_celsius(68) == 20

# GREEN step - simplest possible
class FahrenheitWeather():
    def to_celsius(self, f):
        return 20
```

Beneficios clave

Beneficio	Descripción
Diseño incremental	Foco en interfaces, no en implementación
Baja tasa de bugs	Hasta 40-80% menos según la literatura
Refactorización segura	Se puede cambiar todo con confianza
Documentación viva	Test, doc, y espec all in one

Ciclo completo

Paso detallado

1. Añadir test pequeño
2. Ver que falla (RED)
3. Escribir el código más simple que hace pasar la prueba
4. Validar que pasa (GREEN)

5. Refactorizar manteniendo verde
6. Repetir

Ejemplo de refactor

```
// REFACTOR from GREEN
function fahrenheitToCelsius(f) {
  return 20; // cheating, but GREEN
}
function fahrenheitToCelsius(f) {
  return (f - 32) * 5 / 9;
}
```

Patrones comunes

Patrón	Descripción
Fake It ('til you make it)	Retorna constante falsa, ve generalizando
Triangulate	Con dos ejemplos, infieres la lógica
One to Many	Implementación de una a colecciones
Child Test	Caso padre se resuelve resolviendo hijos

Anti-patrones

- **Gran test**: probar la aplicación completa no es TDD
- **TDD for green (only)**: saltar la refactorización
- **Objetos Mock excesivos**: Acoplamiento a implementaciones
- **Test sin evolucionar**: los que nunca fallan

Herramientas

Language	Framework
Python	pytest, unittest
TypeScript	Jest, Vitest
Java	JUnit 5
Go	testing
C#	xUnit, NUnit

Relación con BDD

BDD es una extensión de TDD que usa lenguaje natural (Given-When-Then) para describir el comportamiento esperado. Ver Behavior driven development.

Métricas asociadas

- Cobertura de código. Relacionado: Metrics/code quality metrics.
- Time to feedback: mide cuánto tarda una iteración TDD desde rojo a verde.
- Número de fallos en PR abiertos: menos de 5% es buena señal.

TDD en CI/CD

Los TDD se ejecutan en cada push. La pipeline CCecha si los test fallan:

```
- name: TDD
  run: pytest tests/ --t da
- name: coverage
  run: pytest --cov=src tests/
```

TDD hace que la confianza del CI sea mucho mayor. Fomenta el refactoring continuo (Refactoring continuo).

Behavior Driven Development

Behavior-Driven Development

Introducción

BDD es una metodología ágil que extiende TDD fomentando la colaboración entre desarrolladores, QA y stakeholders mediante la redacción de casos de prueba en un lenguaje legible en común.

Gherkin Syntax

Gherkin es el lenguaje que permite describir el comportamiento del software en estructuras Given-When-Then.

```
Feature: User login
  Scenario: valid credentials login
    Given a registered user exists with email "user@example.com"
    And the user's password is "secret123"
    When the user logs in with email "user@example.com" and password "secret123"
    Then the system redirects to the dashboard
```

Keywords

Keyword	Descripción
Feature	Título de la funcionalidad
Scenario	Caso específico
Given	Prerequisitos (context)
When	Acción llevada a cabo
Then	Resultado esperado
And / But	Conectores lógicos

Background and Scenario Outline

```
Background:
  Given there is a test database

Scenario Outline: calculator with <a> and <b>
  Given the input is <a> and <b>
  When they are added
  Then the result is <result>
Examples:
  | a | b | result |
```

	1		2		3	
	10		1		11	

Herramientas principales

Language	Tool
Java / Kotlin	Cucumber, JGiven
C# / .NET	SpecFlow
JavaScript	cucumber-js
Python	Behave
Kotlin	JGiven

Idoneidad

BDD es especialmente útil para:

- Reglas de dominio complejo
- Integraciones con sistemas de terceros
- Validación de historias de usuario
- Escenarios largos de flujo completo

BDD no es necesario ni útil para:

- Pruebas unitarias simples
- Lógica puramente algorítmica

Integración con CI

Ejecuta test BDD en integración continua:

```
steps:  
  - run: behave features/
```

Ciclo de vida

1. Definir Feature file con el Product Owner
2. Redactar escenarios (Given-When-Then)
3. Automatizar steps con código
4. Ejecutar tests localmente
5. Agregar a CI suite

Caso de ejemplo completo

Feature: Checkout

As a customer

I want to complete a purchase

So that I can receive my items

Scenario: successful purchase with credit card

Given the user has items in their cart

And the user's credit card is valid

When the user proceeds to checkout

Then the purchase is confirmed

And the user receives an email

BDD vs TDD

Aspecto	TDD	BDD
Audiencia	Desarrolladores	Todos (negocio, QA, Dev)
Lenguaje	Código	Gherkin (legible)
Foco	Componentes	Comportamiento de la aplicación
Herramientas	JUnit, Py Test	Cucumber, Behave

Beneficios

- Acoplar menos la prueba a la implementación
- Documentación siempre viva
- Cobertura de historias de usuario
- Mejor comunicación entre equipo y stakeholders

Ver también

- Test driven development: Fundamentos de TDD
- Documentacion tecnica: Documentación técnica ADR y RFC
- Coverage Metrics

Pair Programming

Pair Programming and Mob Programming

Introducción

Pair programming consiste en dos desarrolladores compartiendo una estación de trabajo: uno escribe código (driver) y el otro revisa en vivo (navigator). Mob programming extiende la idea a todo el equipo, rotando los roles.

El Rol de Driver y Navigator

Rol	Responsabilidades
Driver	Escribe el código, enfocado en los detalles técnicos y sintaxis.
Navigator	Revisa constantemente el código, anticipa errores, plantea enfoques alternativos.

El navegador piensa en la siguiente acción, mientras que el conductor se concentra en la implementación.

Beneficios

Beneficio	Descripción
Reducción de bugs	Dos pares de ojos sobre el mismo fragmento
Transfer de conocimiento	Conocimiento tácito se expresa en voz alta
Mayor calidad de diseño	Las decisiones se discuten mientras se toman
Menos interrupciones	La pareja actúa como una unidad
Mejora en comunicación	Fomenta vocabulario compartido

Pair programming remoto

Herramientas:

- VS Code Live Share
- JetBrains Code With Me
- tmux + tailscale/obico
- ScreenHero / tuple

Para el formato remoto:

- Usar una comunicación de calidad (Mum le voz)
- Rotar el control de la sesión cada 20-30 min.
- Microfono y compartir siempre activados.

Mob programming

En mob programming el equipo completo colabora en una sola tarea.

Reglas del Mob

- 1. Duct schedule: el responsable escribe, los demás observan y sugieren.
- 2. Rotar el conductor cada 10-15 minutos.
- 3. El equipo decide el algoritmo y la estructura.
- 4. Un moderador gestiona la rotación y el tiempo.

Timeline

10:00 – Driver A, rest of team navigators (código checks)
10:15 – Driver B, team navigators
10:30 – Driver C, team navigators

Cuándo usarlo

Context	pair	mob
Bug complejo	X	X
nueva feature	X	X
code review	X	X
refactoring global	X	
diseño arquitectónico	X	
Onboarding	X	X

Errores comunes

- Navegador pasivo: si calla, solo hay un desarrollador. Fomentar el dialogo.
- Driver dictador: el navegador debe tener voz.
- Sesiones demasiado largas: descansar cada 45 min.
- Falta de rotación: un solo par siempre es malo.

Integración con TDD

Pair programming tiene una afinidad natural con TDD. El navegador escribe la prueba (RED), el driver implementa (GREEN), ambos refactorizan (REFACTOR). Ver Test driven development.

Métricas de éxito

- Número de defectos fugados — se reduce en pair programming
- Velocidad del equipo — puede no aumentar inicialmente, pero la calidad si
- Satisfacción del equipo: mejora en equipos complementarios

Ver también Velocity metrics.

Code Review Best Practices

Code Review Best Practices

Introducción

Code reviews sistemáticos reducen bugs y mejoran la consistencia. Deben ser amigables, constructivos y orientados al aprendizaje.

Async vs. Sync reviews

Aspect	Async Review	Sync Review
Modo	Autor escribe, revisor comenta	En vivo, pair style
Herramienta	GitHub/GitLab PRs	Live Share / Team zoom
Ventaja	Tiempo de reflexión	retroalimentación inmediata
Desventaja	Ciclo de retroalimentación largo	Sincronización de agendas
Uso	Cr, features, refactors	Bug fixing, diseño crítico

Review checklist

Category	Items
Design	¿Está diseñado correctamente?
Functionality	¿Hace lo que debe?
Complexity	¿Se puede simplificar?
Tests	¿Son suficiente?
naming	¿Sigue las convenciones?
Comments	¿Son claros y necesarios?
Style	Coincide con guía de estilo?
Documentation	README y ADRs actualizados?

Automation

Usar CI/CD para errores básicos:

Task	Tool
Linting	ruff / eslint / golangci

Task	Tool
Formatting	prettier / black
Secrets scanning	git-secrets / trivy / detect-secrets
Dependency check	Dependabot
Metrics diff	SonarCloud / CodeClimate

How to review

Good practices for the reviewer

- Comenta en tono constructivo. Propón alternativas.
- Diferencia entre "Idea" y "debe cambiarse".
- Usa markdown con estilo.
- Limita a 200-400 lines de código por review.
- Busca bugs, malas prácticas, complejidad innecesaria.

Good practices for the author

- Haz commits pequeños.
- PR menor a 500 líneas.
- Aporta descripción detallada.
- Anticipate preguntas: ¿por qué esta implementación?
- Responde a todos los comentarios.

Metrics for code review

Metric	Target
Review turnaround	< 24 hrs
Lines per PR	< 400
Comment resolution rate	100%
Number of cycles before acceptance	< 3
Author satisfaction	> 80%

Code review in remote teams

Same as Pair programming remote.

Conclusion

Do	Don't
Be kind	Be personal
Provide alternatives	just point out things
Ask questions	Assume knowledge
Focus on correctness	nitpick formatting

Related: Definition of done for DoD.

Documentacion Tecnica

Documentación técnica

Propósito

La documentación técnica sirve como fuente de verdad para decisiones de diseño, API, arquitectura y evolución del sistema.

Tipos de documentos

Tipo	Descripción
ADR	Registro de Decisión de Arquitectura
RFC	Request For Comments (propuesta a revisión)
README	Guía de inicio rápido
API Reference	Documentación generada de código
Architecture docs	C4 / 4+1 views

ADR

Un ADR captura una decisión significativa de arquitectura, el contexto y la consecuencia.

Template

```
# ADR NNNN – Title

## Status
Accepted | Proposed | Superseded

## Context
What is the issue that we're seeing that is motivating this decision?

## Decision
What is the change that we're proposing?

## Consequences
What becomes easier or more difficult?
```

ADR lifecycle

- 1. Draft
- 2. Proposed
- 3. Accepted

4. Superseded

An ADR is never deleted, only superseded.

RFC

Un RFC es un proceso de propuesta. El que la escribe no tiene por qué implementarla.

Phase	Description
Draft	Inicial del autor
Review	Equipo comenta y discute
Decision	Se acepta, rechaza, o pospone
final	RF closes

README best practices

- Título y estado (production / beta / archived)
- Quick start: cómo instalar, configurar, ejecutar.
- Links a documentación larga (wiki / docsify / storybook)
- Badges: CI status, coverage, license

API Reference

Generar documentación directamente del código:

Tool	Language
Swagger/OpenAPI	REST
TypeSpec	API-first
godoc	Go
Javadoc	Java
sphinx	Python

Ver Api design standards.

Architecture docs

Recomendado: C4 model (estatic diagrams as code).

- **Context:** diagrama de sistemas
- **Container:** aplicación y servicios
- **Component:** descomposición interna
- **Code:** (

lectura bajo demanda)

tools: Structurizr, PlantUML.

Documentation as code

- documentación en iposición a código (.md en el repo)
- Se versiona junto al código
- Se actualiza en el mismo PR que el cambio
- Se despliega a doc hub / docsify / vitepress

Audience

Audience	what they need
Developers	API docs, ADRs, RFCs
QA	Test plans, environment docs
Ops	Runbooks, arch docs
Product	Vision docs, RFCs

Related pages

- API Standards
- Git conventions
- Code Review
- DoD

Api Design Standards

AP design standards

Introduction

Proposed:**

Estándar para diseño de APIs REST, GraphQL, gRPC y pautas comunes.

REST

URI naming

Element	Rule
nouns	Use plurals: /users
verbs	Avoid: /getUsers instead use /users
nesting	/users/{id}/orders
cases	kebab-case: /user-profiles

HTTP methods and status

Method	Action	Success code
GET	Read	200
POST	Create	201
PUT	Replace	200
PATCH	Partial update	200
DELETE	Delete	204

Error response body

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid email format",
    "details": [
      {
        "field": "email",
        "issue": "must be valid"
      }
    ]
  }
}
```

```
}  
  ]  
}  
}
```

Pagination

```
GET /users?page=2&per_page=20
```

OpenAPI spec

Define APIs in a machine-readable format:

```
paths:  
  /users:  
    get:  
      summary: List users  
      parameters: [...]  
      responses:  
        "200":  
          description: successful
```

GraphQL

Principles

- Define a clear schema
- Use descriptive names for fields
- Protect against N+1 queries (dataloader)
- Return meaningful error messages
- Versionless (evolve schema)

Mutations

```
type Mutation {  
  createUser(input: CreateUserInput!): User!  
}
```

gRPC

Proto design

- Use semantic naming for messages
- Use enum for finite sets of values
- Use streaming for large data sets

- Use deprecation statuses

```
service UserService {
  rpc GetUser (GetUserRequest) returns (User);
}

message GetUserRequest {
  string user_id = 1;
}
```

API versioning

Approach	Pros	Cons
URL: /v1/users	Easy to route	Polluting URLs
Header: Accept: vnd.api.v1	Clean URL	harder to route
Query: ?version=1	Quick	clutters logs

Security

- Always TLS
- Use API keys or OAuth2
- Rate-limit all endpoints
- Document scopes and permissions
- Ratelimit: X-RateLimit-Remaining

Documentation

- Use a spec language (OpenAPI, TypeSpec)
- Provide interactive playground (Swagger UI)
- Keep changelog up to date
- Link to ADRs for breaking decisions

Ver también

- Documentación ADR
- Coding standards

Git Conventions

Git Conventions

Branches

naming naming

Branch type	naming
feature	feat/IO-100-login
bugfix	fix/IO-201-fix-npe
release	release/v2.1.0
hotfix	hotfix/critical-patch
chore	chore/update-deps

Branching lifecycle

```
main ← release/v2.0.0 ← feat/IO-100
```

1. feat branch from main.
2. PR to main.
3. Squash merge with conventional commit.
4. Delete feature branch.

Conventional Commits

```
<type>(<scope>): <description>

[optional body]
[optional footers]
```

Types

Type	Description
feat	A new feature
fix	A bug fix

Type	Description
docs	Documentation
style	Formatting
refactor	Code improvement
test	Adding/fixing tests
chore	Routine tasks
perf	Performance improvement
ci	CI/CD changes
build	Build/deps

Examples

```
feat(auth): add support for JWT-based API key
fix(stripe): address NPE when customer is undefined
feat(!): drop support for node 12
docs(readme): add Quick start
```

BREAKING: Support removed for node 12

PR templates

Template file at `.github/PULL_REQUEST_TEMPLATE.md` :

```
## Description

## Related Issue

## Type of Change
- [ ] feat
- [ ] fix
- [ ] docs
- [ ] refactor
- [ ] test

## Testing

## Checklist
- [ ] Code follows coding standards
- [ ] Tests pass
- [ ] Documentation updated
- [ ] This PR has <= 400 lines
```

Related: Code review best practices

Merge strategies

Strategy	Used for
Squash merge	Feature branches
Merge commit	release merges
rebase	update feature branches

Git hooks (pre-commit)

Hook	Action
pre-commit	Lint + format
pre-commit	Run unit tests
commit-msg	Validate conventional commit

Git flow Summary

Action	command
start feature	<code>\git checkout -b feat/I0-100-description\</code>
finish feature	PR from feat to main (squash)
fix a bug	<code>fix/IO-xxx-description</code>
create release	<code>release/vMAJOR.MINOR.PATCH</code>

Related

- DoD checklist
- Code review

Refactoring Continuo

Refactoring continuo

Introducción

Refactoring es el proceso de modificar el código interno sin cambiar su comportamiento externo, con el objetivo de mejorar su estructura, legibilidad y mantenibilidad.

Code smells

Smell	Descripción	Solución
Long method	Método con más de 20-30 líneas	Extract method
large class	Clase con múltiples responsabilidades	Extract class
duplicated code	Fragmentos idénticos	Extract method / Use generics
Primitive obsession	usar tipos primitivos en vez de objetos	Replace primitives with value objects
long parameter list	más de 3-4 argumentos	Introduce parameter object
feature envy	un método accede más a datos de otra clase que de la suya	Move method
lazy class	clase que no aporta suficiente	Remove/Inline class
speculative generality	abstracciones innecesarias	Remove hierarchy
message chain	múltiples get() encadenados	Hide delegate
middle man	una clase que solo delega	Remove middle man

Refactoring patterns

pattern	description	applies to
Extract method	Small steps in a method	Almost any code
Rename variable	improve clarity	Many
Move field	from one class to another	Authority fit
Replace conditional with polymorphism	long if/switch chains	OOP
Decompose conditional	split complex condition	Functional

pattern	description	applies to
introduce parameter object	replace long param list	OOP

Example

```
// smelly
double getTotal() {
    double result = price * quantity;
    if (discount > 0) result *= (1 - discount);
    if (tax > 0) result *= (1 + tax);
    if (shipping > 0) result += shipping;
    return result;
}

// refactored
double getSubtotal() { return price * quantity; }
double applyDiscount(double subtotal) { return discount > 0 ? subtotal * (1 - discount) : subtotal; }
double applyTax(double amount) { return tax > 0 ? amount * (1 + tax) : amount; }
double addShipping(double amount) { return amount + shipping; }

double getTotal() {
    return addShipping(applyTax(applyDiscount(getSubtotal())));
}
```

Refactoring cadence

frequency	scale	method
every day	functions/variables	refactory as part of daily TDD cycle
per feature	files/classes	code cleanup in feature branches
per quarter	entire module/component	dedicated refactoring sprint
per milestone	architecture/domain significant	ad-hoc RFC/ADR needed

Tech debt budget

% of sprint	activity
≤ 20%	refactoring debt
≤ 10%	improvement of tooling
≤ 5%	eliminating speculation concerns

The Boy/Girl scout rule

Leave the code better than you found it.

Each change you make, you must improve the code a little.

Important points

Do	Don't
refact only when there is a test	refactor code without tests
commit separately from features	mix refactor with feature commits
prefer small and frequent refactors	plan refactor 2 months ahead

Related

- [Clean Code](#) — Principles supporting refactoring
- [TDD](#) — Safe refactoring foundation
- [Code review](#) — Catches smells (smells)
- [Definition of Done](#) — DoD
- [\[Code Quality\] \(../06-Metricas/code-quality-metrics\)](#)

Definition Of Done

Definition of Done

What is DoD?

Definition of Done is a shared agreement within the team on what it means for a work item to be considered complete. Team defines it together.

Standard object checklist

For every story/PR:

Requirement	Status
Code compiles	✓
Unit tests passed	✓
Linting passes	✓
Integration/API tests passed	✓
Code reviewed and approved	✓
Documentation updated ADR or README	✓
Branch name follows convention (.e.g feat/xxx)	✓
Conventional commit message	✓
PR description is clear	✓
PR is $\leq$ 400 lines	✓
Feature flag added (if toggleable)	✓
Logging added for observability	✓
Monitoring metrics/dashboards set	✓

For a feature:

Requirement	Status
Acceptance criteria met	✓
Tests pass in CI	✓
PO reviewed (if possible)	✓

Requirement	Status
No open PR comments	✓
Breaking changes documented	✓

DoD by artifact type

Artifact	DoD
Epic	DoD defined, all stories accepted, PO signoff
Feature	DoD Defined, all stories accepted, PO signoff
Story	DoD checklist complete, 100% tests passing
PR	All check items complete + reviewed/master approved
Bug	reproduced test added, fix verified on staging

How to create a DoD

1. Hold a team workshop
2. Review past incomplete items
3. Define the minimum criteria
4. Get agreement from whole dev team and product (PO)
5. Write it into the repo (DOCUMENTATION)
6. reference in PR template

why DoD matters

- Eliminates "finished but not done" syndrome
- Creates clear expectations for developers and stakeholders
- Increases transparency about progress
- Reduces the number of open tickets

Anti-pattern

Anti-pattern	Issue
DoD is ignored	Team does not enforce it
DoD is too long	Creates friction, becomes noise
DoD is not reviewed	As the project grows, the DoD must adapt
DoD is owned only by dev	PO and QA must be inside DoD decisions

DoD and technical debt

Related: Refactoring continuo. Without DoD, you can't manage tech debt.

See also

- Code Review – DOD for PRs
- Git conventions – Branch naming, commit style
- Observability – Dr DoD condition for monitoring
- Feature flags – DoD condition for toggled features

Observabilidad Practicas

Observabilidad – Prácticas

Introducción

Observabilidad es la capacidad de inferir el estado interno de un sistema a partir de las señales externas. los tres pilares:

logs, metrics and traces.

The three pillars

Pilar	Descripción
Logs	Eventos individuales, con nivel de severidad
Metrics	Datos agregados sobre comportamiento
Traces	Narrativa de una solicitud a través de un sistema

Logging best practices

Log structure

```
{"timestamp":"2026-07-13T10:00:00Z","level":"WARN","message":"Connection pool exhausted","service":"api-gateway"}
```

Log levels

Level	Use for
ERROR	system is broken
WARN	something unexpected, but handled
INFO	routine important events
DEBUG	verbose, for development
TRACE	extremely detailed steps

rules

- No sensitive information in logs.
- Write structured logs (JSON).
- Use correlation IDs (trace_id) to link logs.

Metrics

- Counters: number of requests, errors
- Gauges: current memory, queue size
- Histograms: latency p50/p95/p99

traces

OpenTemeletry is recommended:

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)
with tracer.start_as_current_span("process-payment"):
    do_payment()
```

OpenTelemetry

Ot is the standard for observability, providing libraries and agents to emit logs, traces and metrics. Itement from application code.

Component	function
OTel API	defines how to generate telemetry
OTel SDK	Implementation
OTel Collector	receive, process, export

Dashboards

Each service should provide a Grafana dashboard:

- Health overview
- Error rate
- p50/p99 latency
- Throughput (RPM/CPM)
- Resource usage (CPU/RAM)

SLO / SLA / Error budget

Concept	definition
SLO	target level of reliability
SL	actual performance contract
Error Budget	allowed window of unavailability

Related: Performance metrics.

Observability in practice

Instrumentation checklist:

action	tool/approach
Log correlation	set trace_id for each request
automated response monitoring	p99 latency alert
dashboard for a new service	SLO initial target (e.g., 99.9%)
4 golden signals per service	latency, traffic, errors, saturation

Post-launch observability

1. Deploy with feature flag
2. Validate logs/metrics/traces make sense
3. adjust thresholds

See also: Feature flags and Chaos engineering.

Related pages

- DoD checklist — Observability as condition for release
- Chaos Engineering — validates observability
- Feature flags — gradual rollout + observability
- Performance metrics — Basis for SLOs

Feature Flags

Feature flags

Introduction

Feature flags (también feature toggles) are a pattern that allows turning functionality on or off without deploying new code. Used for gradual rollouts, A/B testing, and kill switches.

Types of feature flags

Type	lifespan	Description
Release flag	short	hide in-prog. work (trunk-based)
Experiment flag	medium	A/B tests
Ops flag	long	kill switches for prod
Permission flag	indefinite	talk to some org or group

Release flags

Enable trunk-based workflow: merge incomplete features to main, hidden behind a flag.

Ops (Kill) flags

When a dangerous feature needs to be turned off ASAP without redeployment.

Implementation

Simple example (python)

```
from featureflags import FeatureFlag

ff = FeatureFlag()

if ff.is_enabled('new-checkout'):
    use_new_checkout()
else:
    use_old_checkout()
```

Best practices

practice	rationale
Flag config stored in a service (LaunchDarkly, Split)	restartless toggles

practice	rationale
Name flags semantically	e.g. checkout-va-2
always default to off	safe, proactive
audit flag usage	avoid zombie flags
schedule flag removal	prevent flags accumulation

Example config (JSON)

```
{
  "flags": {
    "new-checkout": {
      "enabled": true,
      "users": ["user-abc"]
    },
    "dark-mode": {
      "enabled": false
    }
  }
}
```

Zombie danger

Feature flags that are no longer needed cause cognitive pollution and should be removed. Include a flag removal in the DoD: Definition of done.

LaunchDarkly / Split

Fully managed feature flag as a service:

- serve to specific subsets
- Toggle through web UI
- Run experiments
- kill switches with zero downtime

gradual rollout strategy

1. Enable flag for internal team (canary)
2. Enable for 1% of users
3. Enable for 10%
4. Enable for 100%
5. Monitor performance and errors
6. Remove the flag entirely

Monitoring flag health is critical. See Observabilidad.

flags vs experiment

Feature	Feature flag	Experiment
purpose	release/operate	statistical test
duration	temporary	limited
randomization	not needed	required
Hypothesis	not needed	needed

flags in code review

- Ensure flags default to off
- Review flag naming
- Ensure flag configuration is documented

Related: Code Review.

Summary

Do	Don't
Use for trunk-based dev	leave forever
Remove after release	accumulate zombie flags
Use a service	embed config in code
Document each flag	leave flags without owner

Related Pages

- Observabilidad — log and metric each feature toggle
- DoD — Include flag removal in DoD
- Chaos Engineering — use flag to gradually introduce chaos
- TDD — test toggle state within tests

Chaos Engineering

Chaos Engineering introduction

Chaos engineering is the practice of testing a system by randomly shutting down resources and components to see how it behaves, with the purpose of discovering weaknesses before they cause outages.

Principles

principle	description
Start with baseline	define normal state (e.g., p99 < 100ms)
Hypothesize steady state	predict what should happen
Introduce variedions	e.g., kill a service, latency
measure	Compare actual vs predicted
remediate	fix discovered issues

Game Day

Game Days are planned chaos exercises. The team simulates failure scenarios in a safe environment.

Anatomy of Game Day

Phase	activity
plan	define what to test, what tools to use
execute	run the experiment (using Chaos Monkey, Litmus, etc)
observe	monitor dashboards, SLOs, alerts
analyze	identify gaps in observability/response
improve	create actions for remediation

Scenario examples

scenario	action	Expected observation
Kill a service	stop a pod in Kubernetes	Service mesh redirects to another instance
Add latency	inject 2s delay	p99 spikes, circuit breaker opens
network partition	introduce failure in traffic	eventual consistency may degrade

scenario	action	Expected observation
resource exhaustion	fill up disk	logs management kicks in

Tools

Tool	Use for
Chaos Monkey (Simian Army)	random instance termination
Litmus	kubernetes-native chaos
gremlin	hosted chaos platform
chaos toolkit	customizable experiments
wiremock/pumba	service-level fault injection

Chaos Monkey example

A Chaos Monkey runs at a specified time interval and randomly shuts down production instances (in a fault-tolerant architecture).

Litmus example

```

apiVersion: litmuschaos.io/v1alpha1

kind: ChaosEngine
metadata:
  name: engine-nginx
spec:
  appinfo:
    appns: default
    applabel: app=nginx
    appkind: deployment
  chaosServiceAccount: litmus
  experiments:
    - name: pod-delete
      spec:
        components:
          env:
            - name: TOTAL_CHAOS_DURATION
              value: "30"

```

Observability as a prerequisite

You cannot do Chaos engineering without robust observability. Each experiment must be observable; otherwise you cannot validate the hypothesis.

See Observabilidad practicas.

Blast radius

Run experiments in a controlled fashion, limiting blast radius:

- start in staging or a low impact environment
- Use feature flags for toggle introduce

See Feature flags for gradual rollout.

When to adopt chaos engineering

- your system has at least basic fault tolerance
- you already have CI/CD
- you have monitoring and alerting
- your team has Incident Response processes

See IR.

FAQs

Question	Answer
Is it safe?	Yes, when done gradually.
is it just breaking stuff?	No, it's science: hypothesis, exp, conclusion
can it be continued?	Yes, leading organizations run weekly game days
who should run it?	SRE / platform / senior dev

Related pages

- Observabilidad — prerequisite for chaos
- Feature flags — gradual introduce of chaos
- Definition of done — resilience as part of DoD
- Performance metrics — baseline for experiments
- Incident Response — what happens when the experiment finds a real outage

README

Prácticas y Estándares

Propósito

Esta sección define las prácticas técnicas y los estándares de calidad que todo equipo de desarrollo debe adoptar para mantener un código mantenible, testeable y seguro.

Dimensiones clave

Dimensión	Prácticas asociadas
Calidad de código	Estándares, Clean Code, Refactoring
Testing	TDD, BDD
Colaboración	Pair Programming, Code Review
Documentación	DR, RFC, ADR
Entrega	Definition of Done, Feature Flags

Mapa de la sección

Archivo	Descripción
01-codig-estandar	Estándares de codificación
01-coding-standars	Estándares de cod. (inglés)
02-clean-code	Principios Clean Code
03-tdd	Test-Driven Development
04-bdd	Behavior-Driven Development
05-pair-programming	Pair & Mob Programming
06-code-review	Code Review
07-documentacion-tecnica	Documentación
08-api-standards	API design
09-git-conventions	Convenciones Git
10-refactoring	Refactoring continuo
11-definition-of-done	Definition of Done

Archivo	Descripción
12-observabilidad	Observabilidad
13-feature flags	Feature Flags
14-chaos-engineering	Chaos Engineering

Enlaces de navegación

- ← Fundamentos
- Procesos →
- Herramientas
- Métricas
- Calidad